A MINOR PROJECT REPORT ON

PEER LEARNING PLATFORM

Submitted in partial fulfillment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by:

Saumya Khushlani (Scholar No: 24112011148)
Krishna Gupta (Scholar No: 24112011133)
Shree Pandit (Scholar No: 24112011131)
Kavya Sunil Chanap (Scholar No: 24112011184)

Under the guidance of:

Dr. Bholanath Roy

Department of Computer Science and Engineering
Maulana Azad National Institute of Technology
Bhopal (M.P.) - 462003

April, 2026

Certificate

This is to certify that the project report entitled **"Peer Learning Platform"** submitted by **Saumya Khushlani, Krishna Gupta, Shree Pandit and Kavya Sunil Chanap** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Maulana Azad National Institute of Technology, Bhopal, is a bonafide record of the work carried out by them under my supervision and guidance.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Dr. Bholanath Roy
Project Mentor
CSE Department
MANIT Bhopal

Dr. RK Pateriya
Head of Department
CSE Department
MANIT Bhopal

Declaration

We hereby declare that the minor project report entitled **"Peer Learning Platform"** submitted for the degree of Bachelor of Technology in Computer Science and Engineering at Maulana Azad National Institute of Technology, Bhopal, is our original work and has not been submitted previously in part or full to this or any other university or institution for the award of any degree or diploma.

Saumya Khushlani (24112011148)
Krishna Gupta (24112011133)
Shree Pandit (24112011131)
Kavya Sunil Chanap (24112011184)

Date: April, 2026
Place: MANIT Bhopal

Acknowledgement

We would like to express our deepest appreciation to all those who provided us the possibility to complete this report. A special gratitude we give to our project guide, **Dr. Bholanath Roy**, whose contribution in stimulating suggestions and encouragement helped us to coordinate our project.

Furthermore, we are thankful to the Department of Computer Science and Engineering, MANIT Bhopal, for providing the necessary infrastructure and resources.

Abstract

The Peer Learning Platform is a dedicated, StackOverflow-style Q&A web application engineered for the academic ecosystem. It aims to replace unstructured, ad-hoc doubt resolution channels (like WhatsApp and email threads) with a centralized, searchable, and persistent knowledge base. Built using the PERN stack (PostgreSQL, Express.js, React, Node.js) and Prisma ORM, the platform features tag-based question filtering, markdown-supported rich text editors, nested reply structures, and a reputation-driven voting system.

By crowd-sourcing solutions from peers, the system intends to increase peer-learning engagement by 40% and significantly reduce repetitive queries directed at faculty during office hours. This document details the system architecture, database schema, comprehensive API documentation, and frontend implementation of the platform.

Contents

Certificate **i**

Declaration **ii**

Acknowledgement **iii**

Abstract **iv**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Proposed Solution 1
1.3 Objectives 1
1.4 Leaderboard & Reputation System 2

2 System Architecture **3**
2.1 Technology Stack 3
2.2 Repository Structure 3
2.3 High-Level Application Flow 4

3 Database Design & Modeling **5**
3.1 Entity Relationship Summary 5
3.2 Entity Relationship Diagram 5
3.3 Prisma Schema Definition 6
3.4 Data Definition Language (DDL) Migrations 9

4 Backend API & Service Integration **14**
4.1 Authentication Endpoints 14
4.2 User Management Endpoints 14
4.3 Question Interaction Endpoints 15
4.4 Answer & Reply Endpoints 15
4.5 Voting & Bookmark Endpoints 15

5 Frontend Client Architecture **16**
5.1 API Client & Axios Integration 16

5.1.1 Question Service Abstraction 16
5.2 Core Component Implementation 17
5.2.1 Create Question Interface 17
5.2.2 Landing Interface (Home.jsx) 19

6 System Configurations & Security **22**
6.1 Environment Dependencies 22
6.2 Identified Security Concerns 23

7 Future Scope **24**
7.1 Phase 2 Roadmap 24
7.2 Phase 3 Roadmap 24

List of Tables

2.1 Technology Stack Details 3

Chapter 1

Introduction

1.1 Problem Statement

Students encounter recurring conceptual doubts across core subjects such as Data Structures & Algorithms, Operating Systems, Mathematics, DBMS, and Computer Networks. Existing informal channels offer no structure, no searchability, and no persistence. There is no centralized system that organizes doubts by subject, preserves solutions for future cohorts, or helps faculty identify common struggles.

1.2 Proposed Solution

The Peer Learning Platform is a web application built for the academic ecosystem. It provides students with a structured, searchable space to post academic doubts and receive peer-verified answers. The platform encourages collaborative learning and reduces dependency on offline doubt sessions.

1.3 Objectives

- To increase peer learning engagement by 40%.
- To reduce repetitive queries during faculty office hours by 30%.
- To provide a persistent, searchable knowledge base for all students.
- To implement a reputation system to reward active contributors.

1.4 Leaderboard & Reputation System

Top contributors are recognized through a reputation system to gamify the learning experience. This system incentivizes quality contributions and ensures the most helpful content is surfaced to the top.

- **+10 points:** Accepted Answer (Marked by the original author)
- **+2 points:** Upvote received on a question or answer
- **-1 point:** Downvote received on a question or answer

Chapter 2

System Architecture

2.1 Technology Stack

The application utilizes a modern JavaScript-based full-stack architecture, commonly referred to as the PERN stack, augmented with Prisma as an Object-Relational Mapper (ORM) to ensure type safety and seamless database interactions.

Table 2.1: Technology Stack Details

Layer	Technology	Version
Frontend Framework	React.js	19.x
Frontend Tooling	Vite	8.x
UI Styling	Tailwind CSS	4.x
Markdown Editor	@uiw/react-md-editor	4.x
Backend Runtime	Node.js (ES Modules)	18+
Backend Framework	Express	5.2.x
Database ORM	Prisma	6.19.x
Database Engine	PostgreSQL	14+
Authentication	JWT + bcrypt	9.0.x / 6.0.x

2.2 Repository Structure

The project follows a monorepo-style separation of concerns, clearly dividing the client interface from the server logic. This ensures modularity and independent deployability.

```
peer-doubt-platform/
  client/                  # React + Vite frontend
    src/
      components/          # Shared UI components
      pages/               # Route-level page components
```

```
      lib/api/              # Axios API wrappers per resource
  server/                   # Node.js + Express backend
    src/
      controllers/          # HTTP request handlers
      routes/               # Express Router definitions
      services/             # Business logic + Prisma calls
      middleware/           # Auth middleware
      utils/                # JWT helpers
      config/               # Prisma client singleton
    prisma/
      schema.prisma         # Database schema
      migrations/           # SQL migration history
```

Listing 2.1: Project Directory Layout

2.3 High-Level Application Flow

The application follows a standard client-server architecture with stateless authentication:

1. The React SPA makes authenticated HTTP requests to `/api/*` using Axios.
2. Express routes pass requests through the `authenticate` middleware.
3. The middleware validates the JWT `Authorization: Bearer <token>` header using the `jsonwebtoken` library.
4. If authorized, controllers parse the request payload and delegate to the appropriate service functions.
5. Services execute Prisma ORM queries against the PostgreSQL database.
6. Responses are returned as JSON to the client, triggering asynchronous state updates in React.

Chapter 3

Database Design & Modeling

The database is robustly designed using PostgreSQL. All relationships, constraints, and data types are explicitly defined via the Prisma Schema to ensure referential integrity.

3.1 Entity Relationship Summary

- A **User** is the core entity. It can have many Questions, Answers, Votes, and Bookmarks.
- A **Question** belongs to one User (author) and can have many Answers, Votes, Bookmarks, and Tags.
- An **Answer** belongs to one User and optionally to one Question. Crucially, Answers support nested replies via a self-referential `parentId` foreign key.
- A **Vote** is cast by one User on either a Question or an Answer. Composite unique keys prevent double voting.
- A **Bookmark** is a unique (user, question) pair allowing users to save threads.
- A **Tag** has a unique name. The many-to-many relationship between Questions and Tags is resolved through the **QuestionTag** join table.

3.2 Entity Relationship Diagram

The diagram below visually represents the schema architecture, denoting primary keys, foreign keys, and the cardinality of relationships across the platform's entities.

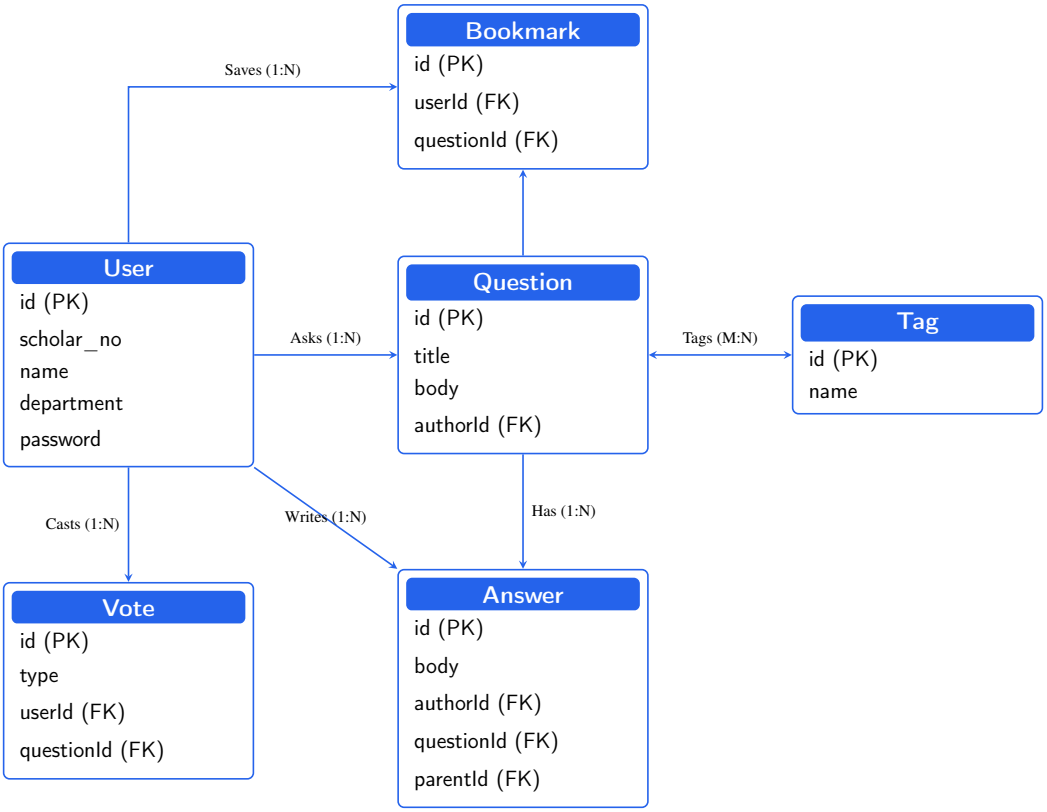

Figure 3.1: Entity Relationship Diagram of the Peer Learning Platform (TikZ)

3.3 Prisma Schema Definition

The entire database schema is centrally defined in the `schema.prisma` file. This is the single source of truth for the database state.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(uuid())
  scholar_no  String    @unique
  name        String
  department  String
  year        Int
  gender      String
```

```
  image       String?
  password    String

  createdAt   DateTime @default(now())

  questions   Question[]
  answers     Answer[]
  votes       Vote[]
  bookmarks   Bookmark[]
}

model Question {
  id          String    @id @default(uuid())
  title       String
  body        String

  authorId    String
  author      User      @relation(fields: [authorId], references
    : [id])

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  answers     Answer[]
  votes       Vote[]
  bookmarks Bookmark[]
  tags        QuestionTag[]
}

model Answer {
  id           String    @id @default(uuid())
  body         String

  authorId     String
  author       User      @relation(fields: [authorId],
    references: [id])

  questionId String?
  question     Question? @relation(fields: [questionId],
    references: [id])

```

```
  parentId   String?
  parent     Answer?   @relation("AnswerReplies", fields: [parentId], references: [id])
  replies    Answer[]  @relation("AnswerReplies")

  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt

  votes      Vote[]
}

enum VoteType {
  UP
  DOWN
}

model Vote {
  id         String   @id @default(uuid())
  type       VoteType

  userId     String
  user       User     @relation(fields: [userId], references: [id])

  questionId String?
  question   Question? @relation(fields: [questionId], references: [id])

  answerId   String?
  answer     Answer?   @relation(fields: [answerId], references: [id])

  createdAt  DateTime @default(now())

  @@unique([userId, questionId])
  @@unique([userId, answerId])
}

model Bookmark {
  id         String   @id @default(uuid())

```

```
  userId     String
  questionId String

  user       User     @relation(fields: [userId], references: [id])
  question   Question @relation(fields: [questionId], references: [id])

  createdAt  DateTime @default(now())

  @@unique([userId, questionId])
}

model Tag {
  id        String   @id @default(uuid())
  name      String   @unique

  questions QuestionTag[]
}

model QuestionTag {
  questionId String
  tagId      String

  question   Question @relation(fields: [questionId], references: [id])
  tag        Tag      @relation(fields: [tagId], references: [id])

  @@id([questionId, tagId])
}
```

Listing 3.1: Prisma Schema Definitions (server/prisma/schema.prisma)

3.4 Data Definition Language (DDL) Migrations

Upon deployment, Prisma compiles the schema into standard SQL to structure the PostgreSQL instance. Below is the primary migration script generated during system initialization.

```
-- CreateEnum
CREATE TYPE "VoteType" AS ENUM ('UP', 'DOWN');
```

```sql

-- CreateTable
CREATE TABLE "User" (
    "id" TEXT NOT NULL,
    "scholar_no" TEXT NOT NULL,
    "name" TEXT NOT NULL,
    "department" TEXT NOT NULL,
    "year" INTEGER NOT NULL,
    "gender" TEXT NOT NULL,
    "image" TEXT NOT NULL,
    "password" TEXT NOT NULL,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
   CURRENT_TIMESTAMP,

    CONSTRAINT "User_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Question" (
    "id" TEXT NOT NULL,
    "title" TEXT NOT NULL,
    "body" TEXT NOT NULL,
    "authorId" TEXT NOT NULL,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
   CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP(3) NOT NULL,

    CONSTRAINT "Question_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Answer" (
    "id" TEXT NOT NULL,
    "body" TEXT NOT NULL,
    "authorId" TEXT NOT NULL,
    "questionId" TEXT,
    "parentId" TEXT,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
   CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP(3) NOT NULL,

```

```
    CONSTRAINT "Answer_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Vote" (
    "id" TEXT NOT NULL,
    "type" "VoteType" NOT NULL,
    "userId" TEXT NOT NULL,
    "questionId" TEXT,
    "answerId" TEXT,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
   CURRENT_TIMESTAMP,

    CONSTRAINT "Vote_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Bookmark" (
    "id" TEXT NOT NULL,
    "userId" TEXT NOT NULL,
    "questionId" TEXT NOT NULL,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
   CURRENT_TIMESTAMP,

    CONSTRAINT "Bookmark_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "Tag" (
    "id" TEXT NOT NULL,
    "name" TEXT NOT NULL,

    CONSTRAINT "Tag_pkey" PRIMARY KEY ("id")
);

-- CreateTable
CREATE TABLE "QuestionTag" (
    "questionId" TEXT NOT NULL,
    "tagId" TEXT NOT NULL,

    CONSTRAINT "QuestionTag_pkey" PRIMARY KEY ("questionId","
```

```
    tagId")
);

-- CreateIndex
CREATE UNIQUE INDEX "User_scholar_no_key" ON "User"("scholar_no");
CREATE UNIQUE INDEX "Vote_userId_questionId_key" ON "Vote"("userId", "questionId");
CREATE UNIQUE INDEX "Vote_userId_answerId_key" ON "Vote"("userId", "answerId");
CREATE UNIQUE INDEX "Bookmark_userId_questionId_key" ON "Bookmark"("userId", "questionId");
CREATE UNIQUE INDEX "Tag_name_key" ON "Tag"("name");

-- AddForeignKey Constraints
ALTER TABLE "Question" ADD CONSTRAINT "Question_authorId_fkey" FOREIGN KEY ("authorId") REFERENCES "User"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Answer" ADD CONSTRAINT "Answer_authorId_fkey" FOREIGN KEY ("authorId") REFERENCES "User"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Answer" ADD CONSTRAINT "Answer_questionId_fkey" FOREIGN KEY ("questionId") REFERENCES "Question"("id") ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE "Answer" ADD CONSTRAINT "Answer_parentId_fkey" FOREIGN KEY ("parentId") REFERENCES "Answer"("id") ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE "Vote" ADD CONSTRAINT "Vote_userId_fkey" FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Vote" ADD CONSTRAINT "Vote_questionId_fkey" FOREIGN KEY ("questionId") REFERENCES "Question"("id") ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE "Vote" ADD CONSTRAINT "Vote_answerId_fkey" FOREIGN KEY ("answerId") REFERENCES "Answer"("id") ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE "Bookmark" ADD CONSTRAINT "Bookmark_userId_fkey" FOREIGN KEY ("userId") REFERENCES "User"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Bookmark" ADD CONSTRAINT "Bookmark_questionId_fkey" FOREIGN KEY ("questionId")
```

```
   REFERENCES "Question"("id") ON DELETE RESTRICT ON UPDATE
   CASCADE;
ALTER TABLE "QuestionTag" ADD CONSTRAINT "
   QuestionTag_questionId_fkey" FOREIGN KEY ("questionId")
   REFERENCES "Question"("id") ON DELETE RESTRICT ON UPDATE
   CASCADE;
ALTER TABLE "QuestionTag" ADD CONSTRAINT "
   QuestionTag_tagId_fkey" FOREIGN KEY ("tagId") REFERENCES "
   Tag"("id") ON DELETE RESTRICT ON UPDATE CASCADE;
```

Listing 3.2: PostgreSQL Database Migration Script

Chapter 4

Backend API & Service Integration

The backend exposes a comprehensive RESTful API for frontend consumption. The base URL for all endpoints is `/api`. All protected endpoints require a valid JWT via the `Authorization: Bearer <token>` HTTP header.

4.1 Authentication Endpoints

POST /api/user/verify
Authenticates an existing user or provisions a new account by dynamically verifying credentials against the official MANIT ERP API.

- **Payload:** `{ "scholar": "2022KUEC2001", "password": "..." }`
- **Response:** Returns a success message, the user profile object, and a signed JWT.
- **Security Note:** The password is encrypted with bcrypt using 10 salt rounds before local persistence. The JWT utilizes the HS256 algorithm with a 7-day expiration.

4.2 User Management Endpoints

GET /api/user/me
Returns the authenticated user's universally unique identifier (UUID) extracted securely from the JWT payload.

GET /api/user/:id
Returns the full user profile, including aggregated statistics such as total questions asked, answers provided, votes cast, and bookmarks saved. Access is rigorously restricted to the token owner.

4.3 Question Interaction Endpoints

POST /api/question

Creates a new question entity. Associated tags are automatically resolved, created if they do not exist, and linked to the question within an atomic database transaction.

- **Payload:** `{ "title": "...", "body": "...", "tags": ["DSA"] }`

GET /api/question

Retrieves a paginated list of questions, utilizing cursor-based pagination (default 15 per page) for optimal performance. Ordered chronologically by `createdAt DESC`.

GET /api/question/:id

Retrieves the comprehensive details of a specific question, joining the author data, associated tags, and nested answers sorted by net vote count.

4.4 Answer & Reply Endpoints

POST /api/answer

Creates a new answer or a nested reply depending on the provided payload parameters.

- **Payload:** `{ "body": "...", "questionId": "uuid", "parentId": null }`
- **Note:** To construct a threaded reply, `parentId` must contain the UUID of the target answer.

4.5 Voting & Bookmark Endpoints

POST /api/vote

Casts or modifies a user's vote on a specific question or answer. The backend service layer handles the "toggle" behavior (e.g., changing an UP vote to a DOWN vote dynamically removes the previous record).

POST /api/bookmark

Saves a specific question to the user's personal bookmark collection for future reference.

DELETE /api/bookmark/question/:questionId

Removes the bookmark association for the authenticated user.

Chapter 5

Frontend Client Architecture

The frontend represents the user interface layer, built as a dynamic Single Page Application (SPA) using React.js.

5.1 API Client & Axios Integration

The application utilizes Axios to abstract all REST API communications. The configuration automatically intercepts requests to attach the necessary JWT authorization headers.

```
import axios from "axios";

export const BASE_URL = import.meta.env.VITE_API_URL ?? "http
    ://localhost:3000";

export function authHeader() {
  const token = localStorage.getItem("token");
  return token ? { Authorization: `Bearer ${token}` } : {};
}

export const api = axios.create({
  baseURL: `${BASE_URL}/api`,
});
```

Listing 5.1: client.js: Axios Configuration

5.1.1 Question Service Abstraction

```
import { api, authHeader } from "./client.js";

```

```
export async function getQuestions({ cursor } = {}) {
  const { data } = await api.get("/question", {
    params: cursor ? { cursor } : {},
    headers: authHeader(),
  });
  return data;
}

export async function getQuestionById(id) {
  const { data } = await api.get(`/question/${id}`, { headers
    : authHeader() });
  return data;
}

export async function getQuestionsByAuthor(id) {
  const { data } = await api.get(`/question/author/${id}`, {
    headers: authHeader(),
  });
  return data;
}
```

Listing 5.2: Question Data Fetching (question.js)

5.2 Core Component Implementation

The React application is modularized into reusable components to ensure maintainability.

5.2.1 Create Question Interface

This component incorporates `@uiw/react-md-editor` to provide students with a robust Markdown editing environment, essential for formatting mathematical formulas and code snippets.

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import MDEditor from "@uiw/react-md-editor";
import "@uiw/react-md-editor/markdown-editor.css";
import { ArrowLeft, HelpCircle, Loader2, Send } from "lucide-
   react";
import { api, authHeader } from "../lib/api/client.js";

```

```
function parseTags(raw) {
  if (!raw || typeof raw !== "string") return [];
  return raw.split(/[,#\s]+/).map((t) => t.trim()).filter(
   Boolean);
}

export default function CreateQuestion() {
  const navigate = useNavigate();
  const [title, setTitle] = useState("");
  const [body, setBody] = useState("");
  const [tagInput, setTagInput] = useState("");
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    const token = localStorage.getItem("token");
    if (!token) {
      navigate("/login");
      return;
    }
    const trimmedTitle = title.trim();
    if (!trimmedTitle) {
      setError("Add a title for your question.");
      return;
    }

    setIsLoading(true);
    try {
      const tags = parseTags(tagInput);
      const { data } = await api.post(
        "/question",
        { title: trimmedTitle, body, tags },
        { headers: { "Content-Type": "application/json", ...
   authHeader() } }
      );
      navigate(`/profile/${data.question.authorId}`);
    } catch (err) {
      setError(err.response?.data?.error ?? "Could not create
    question");
```

```
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="min-h-screen bg-[#f0f7fc] pb-20 font-sans text-slate-900">
      <form onSubmit={handleSubmit} className="space-y-6 p-6 sm:p-8">
        <div>
           <label>Title</label>
           <input type="text" value={title} onChange={(e) => setTitle(e.target.value)} />
        </div>
        <div>
          <label>Details</label>
          <MDEditor value={body} onChange={(v) => setBody(v ?? "")} height={320} preview="live" />
        </div>
        <button type="submit" disabled={isLoading}>
           {isLoading ? <Loader2 className="animate-spin" /> : "Post question"}
        </button>
      </form>
    </div>
  );
}
```

Listing 5.3: CreateQuestion.jsx Component Logic

5.2.2 Landing Interface (Home.jsx)

The Home component introduces the platform's value proposition, emphasizing the structured asking process, crowdsourced truth, and permanent archival of doubts.

```
import { Link } from "react-router-dom";
import { ArrowRight, BookOpen, HelpCircle, MessageSquare, Users, Search, PlusCircle, ShieldCheck, Lock } from "lucide-react";

function isSignedIn() {
  return !!localStorage.getItem("token");
```

```
}

export default function Home() {
  const signedIn = isSignedIn();

  return (
    <div className="min-h-screen bg-white font-sans text-slate-950">
      <section className="relative border-b-4 border-slate-950 px-6 py-16 lg:py-24">
        <div className="mx-auto max-w-7xl relative z-10">
          <h1 className="max-w-4xl text-5xl font-[1000] tracking-tighter sm:text-7xl">
            KNOWLEDGE <br />
            <span className="text-blue-600">UNLOCKED.</span>
          </h1>
          <p className="max-w-xl text-base font-bold text-slate-500 uppercase">
            The internal intelligence hub for MANIT Scholars. Log in with your Scholar Credentials to access the batch archive.
          </p>
          <div className="mt-2 flex flex-col gap-3 sm:flex-row">
            {!signedIn ? (
              <Link to="/login" className="bg-slate-950 text-white px-8 py-3.5">
                <Lock size={18} /> Login to Portal
              </Link>
            ) : (
              <Link to="/question/create" className="bg-blue-600 text-white px-8 py-3.5">
                <PlusCircle size={20} /> New Doubt
              </Link>
            )}
          </div>
        </div>
      </section>
    </div>
  );
}
```

Listing 5.4: Home.jsx Component Selection

Chapter 6

System Configurations & Security

6.1 Environment Dependencies

The backend package execution requires Node.js v18+ and handles server execution scripts natively.

```
{
  "name": "server",
  "version": "1.0.0",
  "main": "src/index.js",
  "scripts": {
    "start": "nodemon src/index.js",
    "start:prod": "node src/index.js"
  },
  "type": "module",
  "dependencies": {
    "@prisma/client": "^6.19.2",
    "axios": "^1.14.0",
    "bcrypt": "^6.0.0",
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3"
  },
  "devDependencies": {
    "nodemon": "^3.1.9",
    "prisma": "^6.19.2"
  }
}
```

Listing 6.1: package.json Configuration Details

6.2 Identified Security Concerns

- **ERP SSL Validation:** Upstream requests to MANIT ERP bypass SSL validation (`rejectUnauthorized: false`). Must be updated for production.
- **CORS Configuration:** Currently uses a wildcard policy; must be restricted to the deployed domain.

Chapter 7

Future Scope

7.1 Phase 2 Roadmap

- Implementation of a real-time notification system.
- Faculty-verified answer tagging capabilities.
- Anonymized posting options for sensitive academic queries.
- Conversion of the frontend into a Progressive Web App (PWA).

7.2 Phase 3 Roadmap

- Deep integration with the MANIT portal via Single Sign-On (SSO).
- AI-driven similar question suggestion during the creation process.
- Advanced gamification including achievement badges and contribution streaks.